

1. What are the pain points in using LLMs?

- a. One of the main pain points we've experienced while using LLMs has been needing to proofread every output to ensure it's both in the correct format and contains coherent information. This became especially painful when generating the 30 use cases as the models output over 30 pages of material, all of which needed to be proofed to ensure the cases fit what we had asked the models to generate. Similarly, when giving the model the use cases from previous parts, the cases needed to be summarized in order to fit them all within the context window. Another pain point we experienced while working with the LLMs was determining how much information we could input or have output by the model before running into issues with the context window. This led to a lot of prompt testing, changing both how many examples were being passed in or how many use cases we could ask to be returned.

2. Any surprises? Eg different conclusions from LLMs?

- a. Overall, there were not many surprises working with the LLMs. Many of the limitations and quirks experienced while working with them were expected behaviors due to our prior experience working with the different models. As for the actual content produced by the models, it was a tad bit surprising that in many cases they chose to think within the box, only creating new ideas after a significant amount of prompting working towards them. This was especially apparent with GPT5 which focused more on fleshing out the material we gave it rather than expanding it to focus on other aspects we might have been missing out on.

3. What worked best?

- a. We found that the LLMs returned the best results when given 1-3 example inputs focused mostly on format and batching outputs in groups of 1-4. When working with Claude, we had to lower these bounds to 1-2 examples and outputs due to the limitations of the context window. We also found that the models worked best when you gave them a predefined list of use case titles to generate from, these of which can also be generated through a chat with the model. This allowed the model to use the thinking portion of the response to work through just the material of the use case rather than needing to also come up with what the use case will be about. While this did require the outputs to be compiled into a doc afterwards, this method still made generating the use cases much easier.

4. What worked worst?

- a. While working through parts a through c, we found that attempting to one-shot the use cases, unsurprisingly, performed extremely poorly, often resulting in either incoherent use cases or not enough cases. This was

especially apparent when working with Claude as it would repeatedly return the same 4 use cases before running out of space in the output. In our larger output batch sizes we found that there would be continuity errors between cases, with some cases containing preconditions which didn't match other similar cases generated in the same batch or preconditions which didn't fit the given context. One of our team members experimented with the use of a locally hosted LLM, and found that despite providing the same input types, the LLM would frequently return false results or not structure output based on the input. This was a universal trait between models showing that locally hosted LLM's are inferior to the larger models that are utilized through the API or chat interfaces.

5. What pre-post processing was useful for structuring the import prompts, then summarizing the output?

- a. One thing we found helpful when preprocessing the material given to the model was removing any special styling characters, new lines, or random spacing created while pasting the examples from Docs. This helped not only make the prompt more human readable but also prevented these oddities from creating artifacts in the LLMs output. This in turn helped decrease the amount of postprocessing which needed to be done in order to get the model's output into the desired format. Using a RAG API also helped improve the quality of the outputs as the models were only given the relevant background information when generating content.

6. Did you find any best/worst prompting strategies?

- a. Overall, we found that detailed prompting with a limited number of example outputs and batching outputs worked best when gathering several similar outputs. For developing ideas, such as use case titles, we found that it was best to introduce the model to the project requirements and description prior to asking for use cases in order to load the project into the models memory, ensuring the context is preserved through the chats. A strategy we avoided due to poor results was zero-shotting as the model tended to not understand the full context of what it was being asked. We also avoided generating large outputs as the models were unable to maintain consistency throughout the output, if the model was able to finish the output prior to running out of space.